# Appendix A Solutions

## GA-Q01

$\lceil 1000000/256 \rceil = 3907$ blocks. The first 3906 cover 999936 elements, leaving 64 useful threads and 192 out-of-range threads in the last block.

The first index is block zero, thread zero, global zero. The last is block 3906, thread 63, global 999999.

Shared memory alone permits $\lfloor 48/12 \rfloor = 4$ blocks. The question explicitly leaves other resources unconstrained. Borrowing the 768-thread limit from GA-Q05 would answer a different question, in which the minimum would instead be three.

## GA-Q02

Two mutually exclusive equal-length paths need two masked passes. Useful lane work is $5 + 27 = 32$, while capacity is $2(32) = 64$, giving 50%.

With no else body, if-body utilization is $5/32 = 15.625\%$. Predication can remove branch/reconvergence overhead but still occupies instruction issue. A false predicate suppresses that lane's operation and must not be counted as a guaranteed memory transaction. The net result depends on code length and target execution rules.

## GA-Q03

Consecutive aligned words occupy 128 bytes, four sectors. Eight-word stride is 32 bytes, placing each lane in a distinct sector: 32 sectors, 1024 transferred sector bytes for 128 useful bytes, or 12.5%.

Starting four bytes into a sector makes the 128-byte interval span five sectors. The first and last are partially useful. Real transaction grouping can differ from this sector count.

DRAM rows, banks, channel balance, cache filtering, latency, and outstanding-request count affect achieved rate. Sector locality does not establish enough concurrency to saturate memory.

## GA-Q04

Consecutive indices visit all banks once, without conflict. Stride 32 uses one bank at 32 different addresses, forming a 32-way conflict under this model.

The exact same address can broadcast, so it need not be a 32-way conflict. Different addresses in one bank are the relevant contention.

# Appendix A Solutions (continued)

Padding a 32-word row stride to 33 can spread transpose accesses across banks. It consumes space and can reduce residency; other phases may have different patterns. Synchronization and actual bank rules must still be measured.

## GA-Q05

| Resource | A's block limit |
|---|---|
| Threads | $768/256 = 3$ |
| Blocks | 8 |
| Registers | $32768/(256 \times 32) = 4$ |
| Shared | $48/16 = 3$ |

The minimum is three, giving 768 threads, 24 warps, and 100% theoretical occupancy.

At 128 threads/block with 16 KiB shared, the limits become six, eight, eight, and three. Three blocks give 384 threads and 50%. With shared use reduced to eight KiB, its limit rises to six, so six blocks restore 768 resident threads.

A 50%-occupancy kernel may exploit more registers, ILP, reuse, or less contention. If enough warps already cover latency, additional warps can add traffic without useful work. Occupancy is a resource count, not a throughput proof.

## GA-Q06

Row zero contains (column 0, value 3) and (2, −1). Row one contains (1, 4). Row two contains (0, 2) and (2, 5).

$$y_0 = 3(2) - 1(3) = 3, \quad y_1 = 4(10) = 40, \quad y_2 = 2(2) + 5(3) = 19.$$

Thus $y = [3, 40, 19]$. With one warp per these short rows, at most 2/32, 1/32, and 2/32 lanes perform useful nonzero work in the first pass. Longer row-length variation also produces unequal completion across warps. In a thread-per-row mapping, unequal row loops instead mask different threads within the same warp.

Repeated indirect reads of x can benefit from reuse. Explicit staging loses if the footprint is large, reuse low, synchronization expensive, or ordinary caches already serve x efficiently. Compare saved traffic against staging cost rather than assuming shared memory is necessary.

# Appendix A Solutions (continued)

## GA-Q07

Ridge intensity is $250/320 = .78125$ FLOP/byte. A's memory ceiling is $.3(320) = 96$ GFLOP/s and its overall ceiling is 96. B's bandwidth product is 1280, so compute limits it to 250.

The purported measurement gives $1.2 \times 10^9/.005 = 240$ GFLOP/s, above A's 96-GFLOP/s ceiling. The supplied inputs cannot all describe the same DRAM-bound phase.

Check whether the intensity counts actual DRAM bytes, whether data are cached, whether timing excludes transfers or a phase, whether FLOP counts include the same executed work, and whether bandwidth units or measurements differ. Divergence, extra transactions, or transfer overhead can reduce performance; they cannot by themselves explain a rate above an otherwise valid upper bound.

For the separate hierarchical phase,

$$I_D = \frac{10^9}{0.25 \times 10^9} = 4, \qquad I_S = \frac{10^9}{4 \times 10^9} = 0.25.$$

$$P_D \le \min(1000, 4(100)) = 400, \quad P_S \le \min(1000, 0.25(1000)) = 250 \quad (\text{GFLOP/s}).$$

$$P \le \min(1000, 400, 250) = 250 \text{ GFLOP/s},$$

$$T_C = 1 \text{ ms}, \quad T_D = 2.5 \text{ ms}, \quad T_S = 4 \text{ ms}, \quad T \ge \max(1, 2.5, 4) = 4 \text{ ms}.$$

DRAM (4,400) and shared (0.25,250) lie on their respective slopes. The shared plateau remains 1000; 250 is the combined bound, not its entire horizontal roof.

## GA-Q08

Useful=8(2)+24(6)=160; issued=32(2+6)=256; utilization=62.5%. The numerator weights each path's active lanes by its length. One-half assumes equal lengths and both paths issued. With B empty and all 32 lanes on A, only two instructions issue, giving 64/64=100%. Kernel/CPU speedup also requires instruction timing, memory waits, synchronization, launch/copies, and a CPU baseline.

## GA-Q09

Addresses 0, 8,.., 248 map to floor (i/4), eight sectors 0..7. Useful 128 bytes/256=50%. Addresses 28+4i span bytes 28..155 and sectors 0..4, five sectors:128/160=80%. Stride 128 gives 32 sectors:128/1024=12.5%. This is one interface request, not necessarily downstream DRAM traffic.

# Appendix A Solutions (continued)

Bank 34 i mod 32=2 i mod 32 has period 16, so 16 banks each serve two distinct words. Bank 35 i mod 32=3 i mod 32 has gcd (3, 32)=1, and all 32 banks differ. Generally the period is 32/gcd (L, 32), and each bank receives gcd (L, 32) distinct words under the stated unswizzled, single-port model.

Padding can cross a residency threshold, such as three blocks becoming two. Extra barriers, worse other-phase layout, and lower occupancy may offset the one-read gain. Compare complete same-workload elapsed time.

## GA-Q10

| Step | Array |
|---|---|
| padded | [2, 0, 1, 3, 4, 0, 0, 0] |
| up 1 | [2, 2, 1, 4, 4, 4, 0, 0] |
| up 2 | [2, 2, 1, 6, 4, 4, 0, 4] |
| up 4 | [2, 2, 1, 6, 4, 4, 0, 10] |
| root 0 | [2, 2, 1, 6, 4, 4, 0, 0] |
| down 4 | [2, 2, 1, 0, 4, 4, 0, 6] |
| down d=2 | [2, 0, 1, 2, 4, 6, 0, 10] |
| down 1 | [0, 2, 2, 3, 6, 10, 10, 10] |

Original outputs are [0, 2, 2, 3, 6], saved total 10. Up/down each use seven additions, total 14, and three levels each, total six, plus constant setup/synchronization. Padding preserves the sum but adds no original elements to output.

The parent prefix p is the sum before its interval. Up-sweep supplies its left subtree total t. The left child needs p; the right child needs p+t. Root prefix 0 is correct because no elements precede it. Induction on depth proves every leaf gets sum (a[0:i]), for all valid integer inputs. Each local block starts at zero, omitting preceding blocks. Scan block sums and add offsets; otherwise the second block's first output is incorrectly 0 rather than the first block sum.

## GA-Q11

z=[0, 0, 0, 1, 0, 0, 1, 1], exclusive Zi=[0, 0, 0, 0, 1, 1, 1, 2], Z=3. Destinations are [3, 4, 5, 0, 6, 7, 1, 2].

# Appendix A Solutions (continued)

| Completed bit | Identity-preserving order |
|---|---|
| 0 | [(0, d),(2, g),(6, h),(5, a),(1, b),(5, c),(1, e),(7, f)] |
| 1 | [(0, d),(5, a),(1, b),(5, c),(1, e),(2, g),(6, h),(7, f)] |
| 2 | [(0, d),(1, b),(1, e),(2, g),(5, a),(5, c),(6, h),(7, f)] |

The jth zero has j zeros before it and goes to j, covering 0..Z−1. The jth one goes to Z+j, covering Z..N−1. Disjoint intervals cover every destination once, so scatter has no write collision. Increasing destinations preserve original order. A later same-high-bit group reversing [0, 1] to [1, 0] would destroy the low-bit invariant; stability is required for the induction. Duplicate identities a/c and b/e retain their order.

## GA-Q12

$$a_0 = 3/2^2 = 3/4, \quad a_1 = -2/2^2 = -1/2, \quad F_0 = 2a_0 = 3/2, \quad F_1 = 3a_1 = -3/2.$$

Total force is zero. Acceleration sum 1/4 is compatible with unequal masses; conservation checks the mass-weighted sum.

Naive sources=$N^2$=64; tiled two blocks each read eight sources, total 16. Add eight my loads to obtain 72 versus 24. At 16 bytes/body, traffic is 1152 versus 384 bytes. Source traffic falls fourfold, total only threefold; do not divide every interface's traffic by four.

For N=10, B=4, last targets 8..11 have only two active outputs, but all four threads load and reach barriers. At source base 8, threads 0/1 load bodies 8/9, threads 2/3 load identities; compute only two valid sources. Early exit can omit required participants or their source loads. Predicate output work rather than skipping collective synchronization.

$$T_{GPU}(K) = 16.777216 + 2.02K \text{ ms}, \qquad T_{CPU} = 25K.$$

$$25K/(16.777216 + 2.02K) \geq 5 \Rightarrow 14.9K \geq 83.88608 \Rightarrow K \geq 5.62994.$$

Minimum K=6. It gives 150/28.897216=5.1908; K=5 gives 125/26.877216=4.6508. Retained data and no overlap are explicit premises; if every iteration copies, the model changes.

# Appendix A Solutions (continued)

## Verification code appendix: gpu_models.py

```
"""Small reproducible models for SIMT divergence, sectors, CSR SpMV and roofline."""

from dataclasses import dataclass


def grid_blocks(elements: int, threads_per_block: int) -> int:
    return (elements + threads_per_block - 1) // threads_per_block


def unique_sectors(words: list[int], sector_bytes: int = 32) -> int:
    return len({(address * 4) // sector_bytes for address in words})


def csr_spmv(ap: list[int], aj: list[int], av: list[int], x: list[float]) -> list[float]:
    if len(ap) < 1:
        raise ValueError("Ap must contain at least one sentinel")
    if ap[0] != 0 or ap[-1] != len(av) or len(aj) != len(av):
        raise ValueError("invalid CSR arrays")
    output = []
    for row in range(len(ap) - 1):
        start, stop = ap[row], ap[row + 1]
        output.append(sum(av[k] * x[aj[k]] for k in range(start, stop)))
    return output


def occupancy_blocks(
    threads_per_block: int,
    registers_per_thread: int,
    shared_kib_per_block: int,
    *,
    sm_threads: int = 768,
    sm_blocks: int = 8,
    sm_registers: int = 32768,
    sm_shared_kib: int = 48,
) -> dict[str, int]:
    return {
        "threads": sm_threads // threads_per_block,
        "blocks": sm_blocks,
        "registers": sm_registers // (threads_per_block * registers_per_thread),
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# Appendix A Solutions (continued)

```
        "shared": sm_shared_kib // shared_kib_per_block,
    }


def main() -> None:
    assert grid_blocks(1_000_000, 256) == 3907
    contiguous = unique_sectors(list(range(32)))
    strided = unique_sectors([8 * lane for lane in range(32)])
    misaligned = unique_sectors([1 + lane for lane in range(32)])
    assert (contiguous, strided, misaligned) == (4, 32, 5)

    y = csr_spmv([0, 2, 3, 5], [0, 2, 1, 0, 2], [3, -1, 4, 2, 5], [2, 10, 3])
    assert y == [3, 40, 19]

    a = occupancy_blocks(256, 32, 16)
    b = occupancy_blocks(128, 32, 16)
    assert min(a.values()) == 3
    assert min(b.values()) == 3
    assert 3 * 256 / 768 == 1.0
    assert 3 * 128 / 768 == 0.5

    memory_roof_a = 320 * 0.3
    bound_a = min(250, memory_roof_a)
    bound_b = min(250, 320 * 4)
    assert (memory_roof_a, bound_a, bound_b) == (96, 96, 250)

    print(f"grid blocks: {grid_blocks(1_000_000, 256)}")
    print(f"sectors contiguous/strided/misaligned: {contiguous}/{strided}/{misaligned}")
    print(f"CSR y: {y}")
    print(f"occupancy A limits: {a}; B limits: {b}")
    print(f"roofline: AI=0.3 -> {bound_a} GFLOP/s; AI=4 -> {bound_b} GFLOP/s")
    print("GPU model checks: PASS")


if __name__ == "__main__":
    main()
```

# Appendix A Solutions (continued)

## Verification code appendix: revision_models.py

```
"""Cost, hierarchy, stack, and coherence models used by the revised manuscripts."""

from dataclasses import dataclass


@dataclass(frozen=True)
class OperationClass:
    count: int
    baseline_cycles_each: float
    local_speedup: float


def cost_breakdown(classes: tuple[OperationClass, ...]) -> tuple[list[float], list[float]]:
    before = [item.count * item.baseline_cycles_each for item in classes]
    after = [time / item.local_speedup for time, item in zip(before, classes)]
    return before, after


@dataclass(frozen=True)
class MemoryBoundary:
    name: str
    bytes_transferred: float
    bytes_per_second: float


def hierarchical_bound(flops: float, compute_flops_per_second: float, boundaries:
tuple[MemoryBoundary, ...]) -> dict:
    levels = [{"name": level.name, "bytes": level.bytes_transferred,
               "intensity": flops / level.bytes_transferred,
               "bound_flops_per_second": flops / level.bytes_transferred *
level.bytes_per_second,
               "minimum_seconds": level.bytes_transferred / level.bytes_per_second}
              for level in boundaries]
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# Appendix A Solutions (continued)

```
    compute_seconds = flops / compute_flops_per_second
    return {"levels": levels, "bound_flops_per_second": min(compute_flops_per_second, *
(level["bound_flops_per_second"] for level in levels)),
            "minimum_seconds": max(compute_seconds, *(level["minimum_seconds"] for level in
levels))}


def p1_cases() -> dict:
    mixed = cost_breakdown((OperationClass(80, 1, 8), OperationClass(20, 10, 1)))
    equal = cost_breakdown((OperationClass(80, 1, 8), OperationClass(20, 1, 1)))
    base = hierarchical_bound(1e9, 1e12, (MemoryBoundary("DRAM", 4e9, 100e9),
MemoryBoundary("shared", 4e9, 1000e9)))
    tiled = hierarchical_bound(1e9, 1e12, (MemoryBoundary("DRAM", .25e9, 100e9),
MemoryBoundary("shared", 4e9, 1000e9)))
    return {"mixed_cost": {"before": mixed[0], "after": mixed[1], "speedup":
sum(mixed[0])/sum(mixed[1])},
            "equal_cost": {"before": equal[0], "after": equal[1], "speedup":
sum(equal[0])/sum(equal[1])},
            "untiled": base, "tiled": tiled,
            "wrong_reused_dram_intensity_bound": min(1e12, 4 * 1000e9)}


if __name__ == "__main__":
    import json
    result = p1_cases()
    assert result["mixed_cost"]["before"] == [80, 200]
    assert result["mixed_cost"]["after"] == [10, 200]
    assert abs(result["mixed_cost"]["speedup"] - 4/3)< 1e-12
    assert abs(result["equal_cost"]["speedup"] - 10/3) < 1e-12
    assert result["untiled"]["bound_flops_per_second"] == 25e9
    assert result["tiled"]["bound_flops_per_second"] == 250e9
    assert result["tiled"]["minimum_seconds"] == .004
    assert result["wrong_reused_dram_intensity_bound"] == 1e12
    print(json.dumps(result, indent=2))
    print("revision cost and hierarchical traffic models: PASS")
```

### References and attribution

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study